How to Install ReactJS on Windows: Step-by-Step Guide

Last Updated : 18 Apr, 2025

ReactJS has become one of the most popular JavaScript libraries for building dynamic and interactive user interfaces, powering some of the biggest websites and applications across the web. If you're a Windows user eager to dive into React development, setting up your environment properly is the first step toward success. Installing ReactJS on Windows may seem daunting at first, but with the right guidance, it's a straightforward process that opens the door to endless possibilities in web development.

In this blog post, we'll walk you through a step-by-step guide to installing ReactJS on your Windows system. From setting up essential tools like Node.js and npm (Node Package Manager) to creating your first React application using Create React App, we'll cover everything you need to get started. Whether you're a beginner or an experienced developer, this guide will help you set up a seamless React development environment tailored for both **Windows 10** and **Windows 11** users.

Prerequisites to Install ReactJS

Before you start setting up React on your Windows machine, it's important to ensure you have the following essentials in place:

Administrator Access

Administrator privileges are required to install software and make system-wide changes on your Windows machine.

- Installing tools like Node.js and npm modifies system files and environment variables.
- Without administrator access, you might face permission errors during the installation process.

*To check if you have administrator access: Right-click on the **Start Menu** and select **Settings > Accounts > Your Info***

Note: If you're using a standard account, you'll need to log in with an admin account or request permissions from the administrator.

Internet Connection

A stable internet connection is required to:

- Download the latest version of **Node.js**, which includes npm, from the official website.
- Fetch React's dependencies during the app creation process.

> ***Tip**: If you experience slow or unstable connectivity, consider downloading offline installers for tools like Node.js or using a faster network to complete the installation process without interruptions.*

System Requirements

Your computer should meet the minimum hardware and software requirements:

- **RAM :** At least 4 GB (8 GB or more is recommended for better performance) 3.
- **Storage :** At least 10 GB of free space on your hard disk to accommodate Node.js, npm, and project files.
- **Operating System :** Windows 10 or 11, macOS 10.10, or Ubuntu 16 (or higher versions) for optimal compatibility and performance

Steps to Install React on Windows

Follow these Steps to Install React.js and Create a React App on Windows 10/11:

Step 1: Install Node.js

React relies on Node.js and its package manager, npm, to manage dependencies. To install React, start by installing **Node.js**, which includes npm (Node Package Manager).

- Visit the official Node.js website.
- Download the **LTS (Long Term Support)** version for Windows. This version is stable and recommended for most users.

> ***Note:*** *Node.js® is a free, open-source, and cross-platform runtime environment for JavaScript. It enables developers to build servers, web applications, command-line tools, and scripts, all using JavaScript outside of a browser.*

Install Node.js

Step 2: **Run the Installer**

- Locate the downloaded `.msi` file and double-click to run it.
- Follow the prompts in the setup wizard, accept the license agreement, and use the default settings for installation.

Step 3: Verify Node.js Installation

Open **Command Prompt** or **PowerShell** > Check the installed versions by running these commands:

- Type `node -v` and press Enter to check the Node.js version.
- Type `npm -v` and press Enter to check the npm version.
- Both commands should return version numbers, confirming successful installation.

Verify Node.js Installation

Step 4: Install CRA (Deprecated Alternative)

Create React App (CRA) has been officially deprecated in 2023 and is no longer the recommended method for starting React projects. Instead, tools like **Vite** or **Next.js** are now preferred for creating React applications due to better performance and modern features. However, if you still want to use it, follow these steps:

1. Install CRA Globally

```
npm install -g create-react-app
```

Install CRA

Additional Tip:

- *However, it's better to switch to newer tools like **Vite**, **Next.js**, or even **CRA alternatives** for a more optimized development experience.*

2. Verify the installation

It will globally install react app for you. To check everything went well run the command

```
create-react-app --version
```

If everything went well it will give you the installed version of react app

Step 5: Create Directory for React Projects

1. Create a New Folder

Create a new folder where you want your React app, use the following command:

```
mkdir newfolder
```

Note: *The newfolder in the above command is the name of the folder and can be anything.*

2. Move inside the folder

Move inside the same folder using the below command:

```
cd newfolder (your folder name)
```

Step 6: Create the React App Using CRA Command

If you're using CRA:

Run the following command to create a React app:

```
npx create-react-app reactfirst
```

Note:

- *The app name must be in lowercase due to npm naming restrictions.*
- *This command installs all required dependencies and sets up your app.*

It will take some time to install the required dependencies

Note: Due to npm naming restrictions, names can no longer contain capital letters, thus type your App name in lowercase.

```
C:\Users\sa552>create-react-app FirstReact
Cannot create a project named "FirstReact" because of npm naming restrictions:

  * name can no longer contain capital letters

Please choose a different project name.
```

If using Vite:

Use the following command

```
npm create vite@latest reactfirst -- --template react
```

Step 7: Open the Project in a Code Editor

Now open the IDE of your choice for eg. Visual studio code and open the folder where you have installed the react app **newfolder** (in the above example) inside the folder you will see your app's name **reactapp** (In our example). Use the terminal and move inside your app name folder.

Use the following command:

```
cd reactapp (your app name)
```

Step 8: Run the React App

Start the development server by running:

```
npm start
```

Once you run the above command, a new browser tab will open, displaying the default React page with the React logo.

Congratulation! You have successfully installed the react-app and are ready to build awesome websites and app. Whether you used CRA, Vite, or another tool, you're now ready to start building dynamic web applications. Explore React's features like components, state, and props to create amazing projects.

Also Read:

- *React Introduction*
- *How to create a website in React JS ?*
- *What are React Components?*

Boost Your React Development Skills with the React JS Course

Take your React development expertise to the next level by enrolling in the ***React JS Course*** *offered by GeeksforGeeks. This complete course offers installation process, covering essential topics like* ***components***, ***state***

***management**, and **routing**. With hands-on projects and real-world applications, you'll gain practical experience to confidently build dynamic, high-performance web applications. Equip yourself with the skills needed to excel in modern web development.*

Conclusion

By following the steps in this guide, you now have a fully functional React development environment on Windows PC or Laptop. Whether you used Create React App (CRA), Vite, or another modern tool, you're ready to start building dynamic, high-performance web applications. React offers endless possibilities for creating interactive user interfaces, and now that your setup is complete, you can explore its core concepts like components, props, state, and hooks. Remember to keep your tools updated and consider using modern frameworks like **Vite** or **Next.js** for future projects to stay aligned with the latest best practices in web development.